Travaux Dirigés Techniques
Framework PHP – Laravel

Niveau intermédiaire

Licence 3 SIO – RAZAFINDRAIBE Fabrice

Projet fil rouge

Application Laravel de gestion de produits, catégories, stocks, utilisateurs et commandes.

$$CRUD \rightarrow Relations \rightarrow Eloquent \rightarrow Sécurité \rightarrow Services \rightarrow Transactions \rightarrow API \rightarrow Tests$$

ESMIA – 2026

Table des matières

1 Organisation pédagogique **2**

2 TD 6 – CRUD complet et validation **3**
2.1 1. Générer un contrôleur resource . . . 3
2.2 2. Déclarer la ressource . . . 3
2.3 3. Mass assignment . . . 4
2.4 4. Créer un Form Request . . . 4
2.5 5. Méthode store() . . . 5
2.6 6. Formulaire de création . . . 5
2.7 7. Tester volontairement des erreurs . . . 7
2.8 8. Validation de la modification . . . 7
2.9 9. Méthode update() . . . 8
2.10 10. Suppression . . . 8
2.11 11. Travail en autonomie . . . 9
2.12 Questions . . . 9

3 TD 7 – Relations Eloquent et intégrité des données **10**
3.1 1. Modèle conceptuel . . . 10
3.2 2. Créer Category . . . 10
3.3 3. Ajouter category_id aux produits . . . 10
3.4 4. Relation Category vers Product . . . 11
3.5 5. Relation inverse . . . 11
3.6 6. Expérimenter avec Tinker . . . 11
3.7 7. Formulaire . . . 12
3.8 8. Affichage . . . 12
3.9 9. Route model binding . . . 12
3.10 10. Travail autonome . . . 13
3.11 11. Analyse . . . 13

4 TD 8 – Eloquent avancé, filtres, pagination et problème N+1 **14**
4.1 1. Cahier des charges . . . 14
4.2 2. Construire la requête . . . 14
4.3 3. Formulaire GET . . . 15
4.4 4. Pagination . . . 15
4.5 5. Créer un scope local . . . 15
4.6 6. Provoquer le problème N+1 . . . 16
4.7 7. Corriger avec eager loading . . . 16
4.8 8. Détecter le lazy loading . . . 16
4.9 9. Travail autonome . . . 17
4.10 10. Analyse . . . 17

5 TD 9 – Factories, Seeders et jeux de données **18**
5.1 1. Pourquoi des données générées ? . . . 18
5.2 2. CategoryFactory . . . 18
5.3 3. ProductFactory . . . 18
5.4 4. Seeder . . . 19
5.5 5. Générer la base . . . 19
5.6 6. Vérifications . . . 20
5.7 7. États de factory . . . 20
5.8 8. Travail autonome . . . 20

6 TD 10 – Authentification, middleware et autorisation **21**
6.1 1. Deux questions différentes . . . 21
6.2 2. Ajouter un statut administrateur . . . 21
6.3 3. Créer un administrateur . . . 21
6.4 4. Protéger des routes . . . 22
6.5 5. Créer une Policy . . . 22
6.6 6. Utiliser la Policy . . . 23
6.7 7. Blade . . . 23
6.8 8. Scénarios de test manuel . . . 24
6.9 9. Question essentielle . . . 24

7 TD 11 – Services, interfaces et injection de dépendances **25**
7.1 1. Situation problématique . . . 25
7.2 2. Créer un service . . . 25
7.3 3. Injection automatique . . . 26
7.4 4. Ajouter une interface . . . 26
7.5 5. Injecter l'interface . . . 27
7.6 6. Binding . . . 27
7.7 7. Analyse . . . 27
7.8 8. Travail autonome . . . 28

8 TD 12 – Transactions et traitement d'une commande **29**
8.1 1. Problème . . . 29
8.2 2. Créer les modèles . . . 29
8.3 3. Migration orders . . . 29
8.4 4. Migration order_items . . . 30
8.5 5. Relations . . . 30
8.6 6. Service de commande . . . 31
8.7 7. Verrouillage du produit . . . 31
8.8 8. Création d'une ligne . . . 32
8.9 9. Stock . . . 32
8.10 10. Expérience rollback . . . 32
8.11 11. Travail autonome . . . 33
8.12 12. Analyse . . . 33

9 TD 13 – Construire une API REST **34**
9.1 1. Activer les routes API . . . 34
9.2 2. Créer le contrôleur . . . 34
9.3 3. Route . . . 34
9.4 4. API Resource . . . 34
9.5 5. index() . . . 35
9.6 6. show() . . . 35
9.7 7. store() . . . 35
9.8 8. destroy() . . . 36
9.9 9. Codes HTTP . . . 36
9.10 10. Tester l'API . . . 36
9.11 11. Travail autonome . . . 36
9.12 12. Comparaison . . . 37

10 TD 14 – Tests automatisés Laravel **38**
10.1 1. Créer un Feature Test . . . 38
10.2 2. Tester le contenu . . . 38
10.3 3. Tester la validation . . . 39
10.4 4. Tester la base . . . 39

10.5 5. Tester l'autorisation . . . 40
10.6 6. Tester l'API . . . 40
10.7 7. Exécuter . . . 41
10.8 8. Test unitaire du service . . . 41
10.9 9. Travail autonome . . . 41

11 TD 15 – Audit technique et refactoring d'une application Laravel 42
11.1 1. Code volontairement problématique . . . 42
11.2 2. Audit individuel . . . 43
11.3 3. Axes possibles . . . 43
11.4 4. Refactoring demandé . . . 43
11.5 5. Deuxième audit : vue Blade . . . 44
11.6 6. Troisième audit : requête . . . 44
11.7 7. Travail final en groupe . . . 45

12 Mini-projet de synthèse 46
12.1 Sujet . . . 46
12.2 Fonctionnalités minimales . . . 46
12.3 Architecture attendue . . . 46

13 Proposition de barème pour le mini-projet 47

14 Grille d'auto-évaluation 47

15 Synthèse finale 47

1. Organisation pédagogique

Cette deuxième série de TD suppose que l'étudiant maîtrise déjà :

— l'installation de Laravel ;
— Composer et Artisan ;
— les routes ;
— les contrôleurs ;
— les vues Blade ;
— les migrations ;
— les modèles Eloquent ;
— les opérations simples de lecture et d'écriture.

L'objectif est maintenant de passer :

de l'utilisation de Laravel

à :

la construction d'une application structurée

La quantité de code fournie diminue progressivement.

TD	Thème	Durée	Guidage
6	CRUD et validation	3h	Fort
7	Relations Eloquent	3h	Fort
8	Eloquent avancé et performance	3h	Moyen
9	Factories et Seeders	2h30	Moyen
10	Authentification et autorisation	3h	Moyen
11	Services et injection de dépendances	3h	Moyen
12	Transactions et commandes	3h	Faible
13	API REST	3h	Faible
14	Tests automatisés	3h	Faible
15	Audit et refactoring	3h	Très faible

2. TD 6 – CRUD complet et validation

Objectifs du TD

À la fin de ce TD, l'étudiant doit pouvoir :

- utiliser un contrôleur resource ;
- utiliser `Route::resource()` ;
- réaliser un CRUD complet ;
- traiter des formulaires HTML ;
- utiliser CSRF et spoofing de méthode ;
- valider les données ;
- créer un Form Request ;
- gérer les erreurs de validation ;
- utiliser `old()` ;
- distinguer validation et mass assignment.

Prérequis

Le modèle `Product` et la table `products` existent déjà.
La table contient au minimum :

id, *name*, *sku*, *price*, *stock*, *description*

2.1. 1. Générer un contrôleur resource

Exécuter :

```
php artisan make:controller ProductController \
--resource --model=Product
```

Examiner les méthodes générées :

- index ;
- create ;
- store ;
- show ;
- edit ;
- update ;
- destroy.

2.2. 2. Déclarer la ressource

Dans `routes/web.php` :

```
use App\Http\Controllers\ProductController;

Route::resource(
    'products',
    ProductController::class
);
```

Exécuter :

```
php artisan route:list
```

Compléter :

Méthode	URI	Action
GET	/products	
GET	/products/create	
POST	/products	
GET	/products/{product}	
GET	/products/{product}/edit	
PUT/PATCH	/products/{product}	
DELETE	/products/{product}	

2.3. 3. Mass assignment

Dans `app/Models/Product.php` :

```
protected $fillable = [
    'name',
    'sku',
    'price',
    'stock',
    'description',
];
```

Questions d'analyse

Pourquoi ne doit-on pas mettre simplement toutes les colonnes de la table dans `$fillable` sans réfléchir ?

2.4. 4. Créer un Form Request

Exécuter :

```
php artisan make:request StoreProductRequest
```

Dans `StoreProductRequest` :

```
public function authorize(): bool
{
    return true;
}

public function rules(): array
{
    return [
        'name' => [
            'required',
            'string',
            'max:150',
        ],

        'sku' => [
            'required',
            'string',
            'max:50',
            'unique:products,sku',
        ],
```

```
21
22          'price' => [
23              'required',
24              'numeric',
25              'min:0',
26          ],
27
28          'stock' => [
29              'required',
30              'integer',
31              'min:0',
32          ],
33
34          'description' => [
35              'nullable',
36              'string',
37          ],
38      ];
39  }
```

2.5. 5. Méthode store()

Dans le contrôleur :

```
use App\Http\Requests\StoreProductRequest;

public function store(
    StoreProductRequest $request
) {
    $product = Product::create(
        $request->validated()
    );

    return redirect()
        ->route('products.show', $product)
        ->with(
            'success',
            'Produit cree avec succes.'
        );
}
```

2.6. 6. Formulaire de création

Créer :

resources/views/products/create.blade.php

```
<x-layout title="Nouveau produit">

<h1>Nouveau produit</h1>

<form
    method="POST"
    action="{{ route('products.store') }}"
>
    @csrf

    <label>Nom</label>
```

```
    <input
        type="text"
        name="name"
        value="{{ old('name') }}"
    >

    @error('name')
        <div>{{ $message }}</div>
    @enderror

    <label>SKU</label>

    <input
        type="text"
        name="sku"
        value="{{ old('sku') }}"
    >

    @error('sku')
        <div>{{ $message }}</div>
    @enderror

    <label>Prix</label>

    <input
        type="number"
        step="0.01"
        name="price"
        value="{{ old('price') }}"
    >

    @error('price')
        <div>{{ $message }}</div>
    @enderror

    <label>Stock</label>

    <input
        type="number"
        name="stock"
        value="{{ old('stock', 0) }}"
    >

    @error('stock')
        <div>{{ $message }}</div>
    @enderror

    <label>Description</label>

    <textarea name="description">{{ old('description') }}</textarea>

    <button type="submit">
        Enregistrer
    </button>

</form>

</x-layout>
```

2.7. 7. Tester volontairement des erreurs

Tester successivement :

1. formulaire vide ;
2. prix négatif ;
3. stock négatif ;
4. stock contenant un texte ;
5. SKU déjà existant ;
6. données correctes.

Point de contrôle

En cas d'erreur :

— l'enregistrement ne doit pas être créé ;
— l'utilisateur doit revenir au formulaire ;
— les messages d'erreur doivent être visibles ;
— les anciennes valeurs doivent être conservées.

2.8. 8. Validation de la modification

Créer :

```
php artisan make:request UpdateProductRequest
```

Dans les règles :

```
use Illuminate\Validation\Rule;

public function rules(): array
{
    return [
        'name' => [
            'required',
            'string',
            'max:150',
        ],

        'sku' => [
            'required',
            'string',
            Rule::unique(
                'products',
                'sku'
            )->ignore(
                $this->route('product')
            ),
        ],

        'price' => [
            'required',
            'numeric',
            'min:0',
        ],

        'stock' => [
            'required',
            'integer',
```

```
            'min:0',
        ],

        'description' => [
            'nullable',
            'string',
        ],
    ];
}
```

2.9. 9. Méthode update()

```
public function update(
    UpdateProductRequest $request,
    Product $product
) {
    $product->update(
        $request->validated()
    );

    return redirect()
        ->route('products.show', $product)
        ->with(
            'success',
            'Produit modifie.'
        );
}
```

2.10. 10. Suppression

Dans Blade :

```
<form
    method="POST"
    action="{{ route(
        'products.destroy',
        $product
    ) }}"
>
    @csrf
    @method('DELETE')

    <button type="submit">
        Supprimer
    </button>
</form>
```

Dans le contrôleur :

```
public function destroy(Product $product)
{
    $product->delete();

    return redirect()
        ->route('products.index')
        ->with(
            'success',
            'Produit supprime.'
        );
}
```

2.11. 11. Travail en autonomie

Travail en autonomie

Terminer les méthodes :

- `index()` ;
- `create()` ;
- `show()` ;
- `edit()`.

Créer également la vue d'édition.
Le formulaire d'édition doit utiliser :

```
@method('PUT')
```

Les valeurs existantes doivent apparaître dans les champs.

2.12. Questions

Questions d'analyse

1. Quelle différence entre POST et PUT ?
2. Pourquoi utiliser `@csrf` ?
3. Pourquoi HTML nécessite-t-il `@method('PUT')` ?
4. Quelle différence entre validation et mass assignment ?
5. Pourquoi utiliser un Form Request ?
6. Quel problème pose un contrôleur contenant 100 lignes de validation ?

Livrable attendu

CRUD fonctionnel de `Product`, avec validation correcte et messages d'erreur.

3. TD 7 – Relations Eloquent et intégrité des données

Objectifs du TD

L'étudiant doit pouvoir :

- créer deux tables liées ;
- utiliser une clé étrangère ;
- définir `hasMany()` ;
- définir `belongsTo()` ;
- exploiter les relations dans Blade ;
- utiliser le route model binding ;
- distinguer relation Eloquent et contrainte relationnelle SQL.

3.1. 1. Modèle conceptuel

Nous voulons représenter :

$$\boxed{\textit{Category } 1 \longrightarrow N \textit{ Product}}$$

Une catégorie possède plusieurs produits.
Un produit appartient au maximum à une catégorie.

3.2. 2. Créer Category

```
php artisan make:model Category -mf
```

Migration :

```
Schema::create(
    'categories',
    function (Blueprint $table) {

        $table->id();

        $table->string('name')
              ->unique();

        $table->timestamps();
    }
);
```

3.3. 3. Ajouter category_id aux produits

```
php artisan make:migration \
add_category_id_to_products_table
```

Migration :

```
Schema::table(
    'products',
    function (Blueprint $table) {

        $table->foreignId(
            'category_id'
        )
```

```
        ->nullable()
        ->constrained()
        ->nullOnDelete();
    }
);
```

Exécuter :

```
php artisan migrate
```

3.4. 4. Relation Category vers Product

Dans `Category.php` :

```
use Illuminate\Database\Eloquent\Relations\HasMany;

public function products(): HasMany
{
    return $this->hasMany(Product::class);
}
```

3.5. 5. Relation inverse

Dans `Product.php` :

```
use Illuminate\Database\Eloquent\Relations\BelongsTo;

public function category(): BelongsTo
{
    return $this->belongsTo(Category::class);
}
```

Ajouter :

```
'category_id',
```

à `$fillable` si ce champ doit être affecté par formulaire.

3.6. 6. Expérimenter avec Tinker

```
php artisan tinker
```

Créer une catégorie :

```
$category = App\Models\Category::create([
    'name' => 'Informatique'
]);
```

Associer un produit :

```
$product = App\Models\Product::first();

$product->category()
        ->associate($category);

$product->save();
```

Tester :

```
$product->category;
```

puis :

```
$category->products;
```

3.7. 7. Formulaire

Ajouter dans les formulaires produit :

```
<select name="category_id">

    <option value="">
        Sans categorie
    </option>

    @foreach($categories as $category)

        <option
            value="{{ $category->id }}"
        >
            {{ $category->name }}
        </option>

    @endforeach

</select>
```

Le contrôleur doit fournir les catégories.
Exemple :

```
$categories = Category::orderBy('name')
    ->get();
```

3.8. 8. Affichage

Dans la fiche produit :

```
<p>
Categorie :
{{ $product->category?->name
    ?? 'Sans categorie' }}
</p>
```

3.9. 9. Route model binding

Observer une méthode telle que :

```
public function show(Product $product)
{
    return view(
        'products.show',
        compact('product')
    );
}
```

Comparer avec :

```
public function show(int $id)
{
    $product = Product::findOrFail($id);

    // ...
}
```

Questions d'analyse

Qu'est-ce que Laravel effectue automatiquement dans la première version ?

3.10. 10. Travail autonome

Travail en autonomie

Créer :

$$\boxed{GET\ /categories/\{category\}}$$

Cette page doit afficher :

- le nom de la catégorie ;
- le nombre de produits ;
- tous les produits appartenant à cette catégorie.

Utiliser une relation Eloquent, et non une requête SQL écrite manuellement.

3.11. 11. Analyse

Questions d'analyse

1. Où se trouve réellement la clé étrangère ?
2. Quelle différence entre `hasMany()` et `belongsTo()` ?
3. Quelle différence entre relation Eloquent et clé étrangère SQL ?
4. Que signifie `nullOnDelete()` ?
5. Quelle autre stratégie serait possible lors de la suppression d'une catégorie ?

4. TD 8 – Eloquent avancé, filtres, pagination et problème N+1

Objectifs du TD

L'étudiant doit pouvoir :

- construire dynamiquement une requête ;
- filtrer selon plusieurs critères ;
- utiliser `when()` ;
- créer un scope local ;
- paginer des résultats ;
- conserver les paramètres de recherche ;
- expliquer le problème N+1 ;
- utiliser l'eager loading.

4.1. 1. Cahier des charges

La page :

`/products`

doit accepter :

`?q=laptop&category=2&min_price=100&available=1`

4.2. 2. Construire la requête

```php
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Http\Request;

public function index(Request $request)
{
    $query = Product::query()
        ->with('category');

    $query->when(
        $request->filled('q'),
        function (Builder $query)
            use ($request) {

            $query->where(
                'name',
                'like',
                '%' .
                $request->string('q') .
                '%'
            );
        }
    );

    $query->when(
        $request->filled('category'),
        function (Builder $query)
            use ($request) {

            $query->where(
```

```
                'category_id',
                $request->integer('category')
            );
        }
    );

    $query->when(
        $request->filled('min_price'),
        function (Builder $query)
            use ($request) {

            $query->where(
                'price',
                '>=',
                $request->input('min_price')
            );
        }
    );

    if ($request->boolean('available')) {
        $query->where('stock', '>', 0);
    }

    $products = $query
        ->orderBy('name')
        ->paginate(10)
        ->withQueryString();

    return view(
        'products.index',
        compact('products')
    );
}
```

4.3. 3. Formulaire GET

Créer un formulaire de filtrage utilisant :

```
<form
    method="GET"
    action="{{ route('products.index') }}"
>
```

Questions d'analyse

Pourquoi un formulaire de recherche utilise-t-il souvent GET plutôt que POST ?

4.4. 4. Pagination

Dans Blade :

```
{{ $products->links() }}
```

Tester avec suffisamment de données.
Vérifier que les filtres restent présents en passant à la page suivante.

4.5. 5. Créer un scope local

Dans le modèle `Product` :

```php
use Illuminate\Database\Eloquent\Attributes\Scope;
use Illuminate\Database\Eloquent\Builder;

#[Scope]
protected function available(
    Builder $query
): void {
    $query->where(
        'stock',
        '>',
        0
    );
}
```

Tester :

```php
Product::available()->get();
```

4.6. 6. Provoquer le problème N+1

Version 1 :

```php
$products = Product::all();
```

Puis dans Blade :

```
@foreach($products as $product)

    {{ $product->name }}

    {{ $product->category?->name }}

@endforeach
```

Analyser conceptuellement les requêtes.
Pour N produits, combien de requêtes peuvent être exécutées ?

4.7. 7. Corriger avec eager loading

Utiliser :

```php
$products = Product::with(
    'category'
)->get();
```

Comparer.

$$\boxed{1 + N \quad \rightarrow \quad \textit{environ 2 requêtes}}$$

4.8. 8. Détecter le lazy loading

En environnement de développement, on peut volontairement rendre visible un mauvais chargement relationnel.

Dans `AppServiceProvider` :

```php
use Illuminate\Database\Eloquent\Model;

public function boot(): void
{
    Model::preventLazyLoading(
```

```
        ! $this->app->isProduction()
    );
}
```

Tester une page contenant un accès à une relation non préchargée.

4.9. 9. Travail autonome

Travail en autonomie

Ajouter les filtres :

- prix maximum ;
- produit en rupture ;
- tri par prix croissant ;
- tri par prix décroissant ;
- tri alphabétique.

Ne jamais concaténer directement un nom de colonne non contrôlé provenant de l'utilisateur dans `orderBy()`.
Utiliser une liste blanche de tris autorisés.

4.10. 10. Analyse

Questions d'analyse

1. Qu'est-ce que le lazy loading ?
2. Qu'est-ce que l'eager loading ?
3. Pourquoi N+1 est-il un problème ?
4. Que fait `with('category')` ?
5. Que fait `withQueryString()` ?
6. Pourquoi utiliser des scopes ?

5. TD 9 – Factories, Seeders et jeux de données

Objectifs du TD

L'étudiant doit pouvoir :

- expliquer l'intérêt des données fictives ;
- créer une Factory ;
- utiliser Faker ;
- créer un Seeder ;
- générer des données relationnelles ;
- reconstruire une base de développement.

5.1. 1. Pourquoi des données générées ?

Tester une pagination, des filtres et N+1 avec trois produits est insuffisant.
Nous voulons par exemple :

$$5\ \textit{categories}$$

et :

$$100\ \textit{produits}$$

5.2. 2. CategoryFactory

Créer si nécessaire :

```
php artisan make:factory CategoryFactory \
--model=Category
```

Exemple :

```
public function definition(): array
{
    return [
        'name' =>
            fake()->unique()->words(
                2,
                true
            ),
    ];
}
```

5.3. 3. ProductFactory

```
php artisan make:factory ProductFactory \
--model=Product
```

```
public function definition(): array
{
    return [
        'name' =>
            fake()->words(3, true),

        'sku' =>
```

```
            fake()
            ->unique()
            ->bothify('PRD-####-??'),

        'price' =>
            fake()->randomFloat(
                2,
                10,
                5000
            ),

        'stock' =>
            fake()->numberBetween(
                0,
                100
            ),

        'description' =>
            fake()->sentence(),

        'category_id' =>
            Category::factory(),
    ];
}
```

5.4. 4. Seeder

Créer :

```
php artisan make:seeder ProductSeeder
```

Exemple :

```
public function run(): void
{
    Category::factory()
        ->count(5)
        ->has(
            Product::factory()
                ->count(20)
        )
        ->create();
}
```

Dans `DatabaseSeeder` :

```
$this->call([
    ProductSeeder::class,
]);
```

5.5. 5. Générer la base

En environnement de développement :

```
php artisan migrate:fresh --seed
```

Attention

`migrate:fresh` détruit les tables existantes.
Cette commande ne doit pas être utilisée sans précaution sur une base contenant des données à conserver.

5.6. 6. Vérifications

Avec Tinker :

```
Product::count();
```

```
Category::count();
```

```
Category::withCount('products')->get();
```

5.7. 7. États de factory

Créer un état permettant de générer un produit en rupture :

```
public function outOfStock(): static
{
    return $this->state(
        fn () => [
            'stock' => 0,
        ]
    );
}
```

Tester :

```
Product::factory()
    ->outOfStock()
    ->count(10)
    ->create();
```

5.8. 8. Travail autonome

Travail en autonomie

Produire une base contenant exactement :

- 6 catégories ;
- 120 produits ;
- au moins 20 produits en rupture ;
- au moins 20 produits ayant un prix supérieur à 2000.

Vérifier les nombres avec Eloquent.

6. TD 10 – Authentification, middleware et autorisation

Objectifs du TD

L'étudiant doit pouvoir :

- distinguer authentification et autorisation ;
- utiliser le middleware `auth` ;
- protéger un groupe de routes ;
- créer une Policy ;
- contrôler création, modification et suppression ;
- utiliser `@can` dans Blade.

Prérequis

Ce TD suppose que le projet dispose déjà du mécanisme Laravel de connexion et déconnexion d'un utilisateur.
Le TD porte principalement sur la protection des fonctionnalités et non sur la conception HTML de l'écran de connexion.

6.1. 1. Deux questions différentes

$$\boxed{Authentification : Qui êtes-vous ?}$$

$$\boxed{Autorisation : Avez-vous le droit ?}$$

6.2. 2. Ajouter un statut administrateur

Créer une migration :

```
php artisan make:migration \
add_is_admin_to_users_table
```

```
Schema::table(
    'users',
    function (Blueprint $table) {

        $table->boolean('is_admin')
              ->default(false);
    }
);
```

Exécuter :

```
php artisan migrate
```

6.3. 3. Créer un administrateur

Avec Tinker :

```
$user = App\Models\User::first();

$user->is_admin = true;

$user->save();
```

6.4. 4. Protéger des routes

Les visiteurs peuvent consulter.
Les utilisateurs authentifiés peuvent atteindre les actions d'écriture.
Exemple :

```
Route::resource(
    'products',
    ProductController::class
)->only([
    'index',
    'show',
]);

Route::middleware('auth')
    ->group(function () {

        Route::resource(
            'products',
            ProductController::class
        )->except([
            'index',
            'show',
        ]);
    });
```

6.5. 5. Créer une Policy

```
php artisan make:policy ProductPolicy \
--model=Product
```

Dans `ProductPolicy` :

```
public function viewAny(
    User $user
): bool {
    return true;
}

public function view(
    User $user,
    Product $product
): bool {
    return true;
}

public function create(
    User $user
): bool {
    return $user->is_admin;
}

public function update(
    User $user,
    Product $product
): bool {
    return $user->is_admin;
}

public function delete(
```

```
    User $user,
    Product $product
): bool {
    return $user->is_admin;
}
```

6.6. 6. Utiliser la Policy

Dans le contrôleur :

```
use Illuminate\Support\Facades\Gate;

public function edit(Product $product)
{
    Gate::authorize(
        'update',
        $product
    );

    // ...
}
```

Même principe pour :

- create ;
- store ;
- update ;
- destroy.

6.7. 7. Blade

Exemple :

```
@can('create', App\Models\Product::class)

    <a href="{{ route(
        'products.create'
    ) }}">
        Nouveau produit
    </a>

@endcan
```

Pour une ressource :

```
@can('update', $product)

    <a href="{{ route(
        'products.edit',
        $product
    ) }}">
        Modifier
    </a>

@endcan
```

6.8. 8. Scénarios de test manuel

Tester avec :

1. visiteur non connecté ;
2. utilisateur normal ;
3. administrateur.

Compléter :

Action	Visiteur	User	Admin
Voir catalogue			
Voir produit			
Créer			
Modifier			
Supprimer			

6.9. 9. Question essentielle

Questions d'analyse

Pourquoi cacher le bouton “Supprimer” avec Blade ne constitue-t-il pas à lui seul une véritable sécurité ?

7. TD 11 – Services, interfaces et injection de dépendances

Objectifs du TD

L'étudiant doit pouvoir :

- reconnaître un contrôleur trop chargé ;
- extraire la logique métier ;
- créer une classe Service ;
- injecter une dépendance ;
- comprendre la résolution automatique ;
- utiliser une interface ;
- créer un binding dans le Service Container.

7.1. 1. Situation problématique

Un contrôleur calcule :

- sous-total ;
- remise ;
- taxe ;
- total final.

On trouve alors :

```php
$total = 0;

foreach ($items as $item) {

    $subtotal =
        $item['price']
        * $item['quantity'];

    $total += $subtotal;
}

if ($total > 1000) {
    $total *= 0.95;
}

$total *= 1.20;
```

Questions d'analyse

Ces règles appartiennent-elles réellement au traitement HTTP ?

7.2. 2. Créer un service

Créer :

`app/Services/OrderTotalService.php`

```php
<?php

namespace App\Services;
```

```
4
5  class OrderTotalService
6  {
7      public function calculate(
8          array $items
9      ): float {
10         $total = 0;
11
12         foreach ($items as $item) {
13
14             $total +=
15                 $item['price']
16                 * $item['quantity'];
17         }
18
19         if ($total > 1000) {
20             $total *= 0.95;
21         }
22
23         return round(
24             $total * 1.20,
25             2
26         );
27     }
28 }
```

7.3. 3. Injection automatique

Dans le contrôleur :

```
use App\Services\OrderTotalService;

public function __construct(
    private OrderTotalService
        $totalService
) {
}
```

Puis :

```
$total =
    $this->totalService
         ->calculate($items);
```

Point de contrôle

Aucune instruction `new OrderTotalService()` n'est nécessaire dans le contrôleur.
Laravel résout automatiquement la dépendance concrète.

7.4. 4. Ajouter une interface

Créer :

`app/Contracts/OrderTotalCalculator.php`

```
<?php

namespace App\Contracts;

```

```php
interface OrderTotalCalculator
{
    public function calculate(
        array $items
    ): float;
}
```

Modifier le service :

```php
class OrderTotalService
    implements OrderTotalCalculator
{
    // ...
}
```

7.5. 5. Injecter l'interface

Dans le contrôleur :

```php
use App\Contracts\OrderTotalCalculator;

public function __construct(
    private OrderTotalCalculator
        $calculator
) {
}
```

À ce stade Laravel doit savoir quelle implémentation utiliser.

7.6. 6. Binding

Dans `AppServiceProvider` :

```php
use App\Contracts\OrderTotalCalculator;
use App\Services\OrderTotalService;

public function register(): void
{
    $this->app->bind(
        OrderTotalCalculator::class,
        OrderTotalService::class
    );
}
```

7.7. 7. Analyse

Questions d'analyse

1. Quel est le rôle du contrôleur ?
2. Quel est le rôle du service ?
3. Pourquoi ne pas instancier le service directement avec `new` ?
4. Quel avantage apporte une interface ?
5. Quel est le rôle du Service Container ?

7.8. 8. Travail autonome

Travail en autonomie

Créer un service :

`StockAvailabilityService`

avec une méthode :

```
public function canFulfill(
    Product $product,
    int $quantity
): bool
```

Le contrôleur ne doit plus contenir directement la règle :

```
$product->stock >= $quantity
```

8. TD 12 – Transactions et traitement d'une commande

Objectifs du TD

L'étudiant doit pouvoir :

- modéliser une opération métier multi-étapes ;
- créer Order et OrderItem ;
- utiliser une transaction ;
- comprendre commit et rollback ;
- contrôler les stocks ;
- utiliser `lockForUpdate()` ;
- préserver la cohérence des données.

8.1. 1. Problème

Passer une commande signifie :

$$\boxed{\textit{Créer Order}}$$

puis :

$$\boxed{\textit{Créer OrderItems}}$$

puis :

$$\boxed{\textit{Décrémenter les stocks}}$$

Que se passe-t-il si une erreur intervient entre la première et la dernière opération ?

8.2. 2. Créer les modèles

```
php artisan make:model Order -m
php artisan make:model OrderItem -m
```

8.3. 3. Migration orders

```
Schema::create(
    'orders',
    function (Blueprint $table) {

        $table->id();

        $table->foreignId('user_id')
              ->constrained()
              ->cascadeOnDelete();

        $table->decimal(
            'total',
            12,
            2
        )->default(0);

        $table->string('status')
              ->default('pending');
```

```
19
20          $table->timestamps();
21      }
22  );
```

8.4. 4. Migration order_items

```
1   Schema::create(
2       'order_items',
3       function (Blueprint $table) {
4
5           $table->id();
6
7           $table->foreignId('order_id')
8                 ->constrained()
9                 ->cascadeOnDelete();
10
11          $table->foreignId('product_id')
12                ->constrained()
13                ->restrictOnDelete();
14
15          $table->unsignedInteger(
16              'quantity'
17          );
18
19          $table->decimal(
20              'unit_price',
21              12,
22              2
23          );
24
25          $table->decimal(
26              'subtotal',
27              12,
28              2
29          );
30
31          $table->timestamps();
32      }
33  );
```

8.5. 5. Relations

Dans `Order` :

```
1   public function items(): HasMany
2   {
3       return $this->hasMany(
4           OrderItem::class
5       );
6   }
7
8   public function user(): BelongsTo
9   {
10      return $this->belongsTo(
11          User::class
12      );
13  }
```

Dans `OrderItem` :

```php
public function order(): BelongsTo
{
    return $this->belongsTo(
        Order::class
    );
}

public function product(): BelongsTo
{
    return $this->belongsTo(
        Product::class
    );
}
```

8.6. 6. Service de commande

Créer :

`app/Services/OrderService.php`

Structure générale :

```php
use Illuminate\Support\Facades\DB;
use Illuminate\Validation\ValidationException;

public function place(
    User $user,
    array $items
): Order {

    return DB::transaction(
        function () use (
            $user,
            $items
        ) {

            // 1. Creer la commande

            // 2. Charger et verrouiller
            // les produits

            // 3. Verifier les stocks

            // 4. Creer les lignes

            // 5. Decrementer les stocks

            // 6. Calculer le total

            // 7. Retourner la commande
        }
    );
}
```

8.7. 7. Verrouillage du produit

Pour chaque ligne :

```php
$product = Product::query()
    ->lockForUpdate()
    ->findOrFail(
        $item['product_id']
    );
```

Vérification :

```php
if (
    $product->stock
    < $item['quantity']
) {
    throw ValidationException
        ::withMessages([
            'stock' =>
            "Stock insuffisant pour "
            . $product->name,
        ]);
}
```

8.8. 8. Création d'une ligne

```php
$subtotal =
    $product->price
    * $item['quantity'];

$order->items()->create([
    'product_id' =>
        $product->id,

    'quantity' =>
        $item['quantity'],

    'unit_price' =>
        $product->price,

    'subtotal' =>
        $subtotal,
]);
```

8.9. 9. Stock

```php
$product->decrement(
    'stock',
    $item['quantity']
);
```

8.10. 10. Expérience rollback

Introduire volontairement :

```php
throw new \Exception(
    'Erreur volontaire'
);
```

après la création de la première ligne.
Observer la base.

Point de contrôle

Aucune partie de l'opération ne doit rester enregistrée si la transaction échoue.

8.11. 11. Travail autonome

Travail en autonomie

Terminer complètement la méthode `place()`.
Contraintes :

- aucun stock négatif ;
- une commande sans ligne est interdite ;
- le total doit correspondre à la somme des sous-totaux ;
- une erreur doit provoquer un rollback ;
- le contrôleur doit rester court.

8.12. 12. Analyse

Questions d'analyse

1. Qu'est-ce qu'une transaction ?
2. Que signifie atomicité ?
3. Qu'est-ce qu'un rollback ?
4. Pourquoi verrouiller le produit ?
5. Quelle incohérence serait possible sans transaction ?

9. TD 13 – Construire une API REST

Objectifs du TD

L'étudiant doit pouvoir :

- expliquer REST ;
- créer des routes API ;
- utiliser `apiResource()` ;
- retourner du JSON ;
- utiliser les codes HTTP ;
- créer une API Resource ;
- valider une requête API ;
- distinguer interface Web et API.

9.1. 1. Activer les routes API

Si le projet ne possède pas encore `routes/api.php` :

```
php artisan install:api
```

9.2. 2. Créer le contrôleur

```
php artisan make:controller \
Api/ProductController \
--api --model=Product
```

9.3. 3. Route

Dans `routes/api.php` :

```
use App\Http\Controllers\Api\ProductController;

Route::apiResource(
    'products',
    ProductController::class
);
```

Vérifier :

```
php artisan route:list
```

9.4. 4. API Resource

Créer :

```
php artisan make:resource ProductResource
```

Exemple :

```
public function toArray(
    Request $request
): array {
    return [
        'id' =>
            $this->id,
```

```
7
8           'name' =>
9               $this->name,
10
11          'sku' =>
12              $this->sku,
13
14          'price' =>
15              $this->price,
16
17          'stock' =>
18              $this->stock,
19
20          'category' =>
21              $this->category?->name,
22      ];
23  }
```

9.5. 5. index()

```
1   public function index()
2   {
3       $products = Product::with(
4           'category'
5       )->paginate(20);
6
7       return ProductResource::collection(
8           $products
9       );
10  }
```

9.6. 6. show()

```
1   public function show(
2       Product $product
3   ) {
4       $product->load('category');
5
6       return new ProductResource(
7           $product
8       );
9   }
```

9.7. 7. store()

Créer une validation adaptée ou réutiliser un Form Request.
Exemple de retour :

```
1   $product = Product::create(
2       $request->validated()
3   );
4
5   return (
6       new ProductResource($product)
7   )->response()
8    ->setStatusCode(201);
```

9.8. 8. destroy()

```php
public function destroy(
    Product $product
) {
    $product->delete();

    return response()->noContent();
}
```

9.9. 9. Codes HTTP

Associer :

Code	Signification
200	
201	
204	
401	
403	
404	
422	
500	

9.10. 10. Tester l'API

Utiliser :

— navigateur pour GET ;

— curl ;

— Postman ou outil équivalent.

Exemple :

```
curl http://localhost:8000/api/products
```

9.11. 11. Travail autonome

Travail en autonomie

Terminer les cinq opérations :

$$GET,\ POST,\ GET/id,\ PUT/PATCH,\ DELETE$$

Exigences :

— JSON uniquement ;

— validation ;

— bons codes HTTP ;

— relations correctement chargées ;

— pas de HTML dans le contrôleur API.

9.12. 12. Comparaison

Questions d'analyse

Compléter conceptuellement :

$$Controller\ Web \rightarrow \boxed{?}$$

$$Controller\ API \rightarrow \boxed{?}$$

Le modèle Eloquent doit-il nécessairement être différent ?

10. TD 14 – Tests automatisés Laravel

Objectifs du TD

L'étudiant doit pouvoir :

- expliquer l'intérêt d'un test automatisé ;
- distinguer test unitaire et test fonctionnel ;
- utiliser une base de test ;
- utiliser `RefreshDatabase` ;
- tester une réponse HTTP ;
- tester la validation ;
- tester la base ;
- tester une autorisation ;
- tester une API.

10.1. 1. Créer un Feature Test

```
php artisan make:test ProductTest
```

Exemple PHPUnit :

```php
<?php

namespace Tests\Feature;

use Tests\TestCase;
use App\Models\Product;
use Illuminate\Foundation\Testing\RefreshDatabase;

class ProductTest extends TestCase
{
    use RefreshDatabase;

    public function test_products_page_is_accessible(): void
    {
        $response =
            $this->get('/products');

        $response->assertStatus(200);
    }
}
```

10.2. 2. Tester le contenu

```php
public function test_product_is_visible(): void
{
    $product =
        Product::factory()->create([
            'name' =>
                'Laptop Test',
        ]);

    $response =
        $this->get('/products');
```

```
11
12      $response
13          ->assertOk()
14          ->assertSeeText(
15              'Laptop Test'
16          );
17  }
```

10.3. 3. Tester la validation

```
1   public function test_product_name_is_required(): void
2   {
3       $admin =
4           User::factory()->create([
5               'is_admin' => true,
6           ]);
7
8       $response =
9           $this->actingAs($admin)
10              ->post(
11                  route(
12                      'products.store'
13                  ),
14                  [
15                      'name' => '',
16                      'sku' => 'TEST-1',
17                      'price' => 100,
18                      'stock' => 5,
19                  ]
20              );
21
22      $response->assertInvalid([
23          'name',
24      ]);
25  }
```

10.4. 4. Tester la base

```
1   public function test_admin_can_create_product(): void
2   {
3       $admin =
4           User::factory()->create([
5               'is_admin' => true,
6           ]);
7
8       $category =
9           Category::factory()->create();
10
11      $response =
12          $this->actingAs($admin)
13              ->post(
14                  route(
15                      'products.store'
16                  ),
17                  [
18                      'name' =>
19                          'New Product',
20
```

```
                    'sku' =>
                        'NEW-001',

                    'price' =>
                        500,

                    'stock' =>
                        10,

                    'category_id' =>
                        $category->id,
                ]
            );

    $response->assertSessionHasNoErrors();

    $this->assertDatabaseHas(
        'products',
        [
            'sku' => 'NEW-001',
        ]
    );
}
```

10.5. 5. Tester l'autorisation

```
public function test_normal_user_cannot_delete_product(): void
{
    $user =
        User::factory()->create([
            'is_admin' => false,
        ]);

    $product =
        Product::factory()->create();

    $response =
        $this->actingAs($user)
             ->delete(
                route(
                    'products.destroy',
                    $product
                )
            );

    $response->assertForbidden();

    $this->assertDatabaseHas(
        'products',
        [
            'id' => $product->id,
        ]
    );
}
```

10.6. 6. Tester l'API

```
public function test_api_returns_products(): void
```

```
{
    Product::factory()
        ->count(3)
        ->create();

    $response =
        $this->getJson(
            '/api/products'
        );

    $response
        ->assertOk()
        ->assertJsonStructure([
            'data',
        ]);
}
```

10.7. 7. Exécuter

```
php artisan test
```

Pour un seul fichier :

```
php artisan test \
tests/Feature/ProductTest.php
```

10.8. 8. Test unitaire du service

Créer :

```
php artisan make:test \
OrderTotalServiceTest --unit
```

Tester par exemple :

$$2 \times 100 + 1 \times 300$$

Le résultat calculé doit correspondre à la règle métier définie.

10.9. 9. Travail autonome

Travail en autonomie

Écrire au minimum les tests suivants :

1. liste des produits accessible ;
2. produit inexistant retourne 404 ;
3. nom obligatoire ;
4. SKU doit être unique ;
5. prix négatif interdit ;
6. utilisateur normal ne peut supprimer ;
7. administrateur peut créer ;
8. API index retourne du JSON ;
9. suppression API retourne 204 ;
10. calcul du total d'une commande est correct.

11. TD 15 – Audit technique et refactoring d'une application Laravel

Objectifs du TD

Ce dernier TD cherche moins à apprendre une nouvelle commande qu'à évaluer la capacité de l'étudiant à analyser une application existante.
L'étudiant doit pouvoir :

- identifier les responsabilités mal placées ;
- détecter un problème de sécurité ;
- détecter un problème N+1 ;
- détecter une absence de validation ;
- reconnaître un contrôleur trop chargé ;
- proposer une architecture améliorée ;
- refactoriser sans changer le comportement attendu.

11.1. 1. Code volontairement problématique

Étudier :

```php
public function store(Request $request)
{
    if ($request->user()->is_admin == 0) {
        abort(403);
    }

    $product =
        Product::create(
            $request->all()
        );

    if ($request->category) {

        $product->category_id =
            $request->category;

        $product->save();
    }

    $products = Product::all();

    foreach ($products as $p) {

        echo $p->category->name;
    }

    if ($product->stock < 0) {

        $product->stock = 0;

        $product->save();
    }

    return '<h1>Produit cree</h1>';
}
```

11.2. 2. Audit individuel

Identifier au minimum **huit problèmes**.
Pour chacun :

N°	Problème	Correction proposée
1		
2		
3		
4		
5		
6		
7		
8		

11.3. 3. Axes possibles

L'analyse doit notamment examiner :

— validation ;
— mass assignment ;
— autorisation ;
— Policy ;
— relations ;
— eager loading ;
— séparation HTTP / métier ;
— Blade ;
— réponse HTTP ;
— qualité des noms ;
— duplication ;
— testabilité.

11.4. 4. Refactoring demandé

Proposer une architecture dans laquelle :

Route

appelle :

Controller

qui utilise :

FormRequest

et :

Policy

puis éventuellement :

Service

qui utilise :

Eloquent

avant de retourner :

View ou *JSON*

11.5. 5. Deuxième audit : vue Blade

Code :

```
@foreach($products as $product)

    <h2>{{ $product->name }}</h2>

    <p>
        {{ $product->category->name }}
    </p>

    @if(auth()->user()->is_admin)

        <form
            method="POST"
            action="/products/{{ $product->id }}"
        >
            <button>
                Supprimer
            </button>
        </form>

    @endif

@endforeach
```

Identifier les problèmes potentiels.

11.6. 6. Troisième audit : requête

Code :

```
$products = Product::where(
    'name',
    'like',
    '%' . $_GET['q'] . '%'
)->get();
```

Proposer une version plus cohérente avec Laravel.

11.7. 7. Travail final en groupe

Travail en autonomie

Par groupes de 3 à 4 étudiants :

1. analyser le projet Laravel réalisé pendant les TD précédents ;
2. identifier au moins dix améliorations ;
3. classer les problèmes en :
   - architecture ;
   - données ;
   - sécurité ;
   - performances ;
   - maintenabilité ;
   - tests.
4. sélectionner cinq corrections prioritaires ;
5. réaliser les corrections ;
6. préparer une justification technique.

12. Mini-projet de synthèse

Après les TD 6 à 15, l'étudiant doit être capable de réaliser une petite application Laravel complète.

12.1. Sujet

Développer :

Application de gestion de stock et de commandes

L'application doit contenir :

— utilisateurs ;
— catégories ;
— produits ;
— commandes ;
— lignes de commande.

12.2. Fonctionnalités minimales

1. authentification ;
2. catalogue paginé ;
3. recherche multicritère ;
4. CRUD produit ;
5. CRUD catégorie ;
6. relations Eloquent ;
7. autorisations ;
8. création d'une commande ;
9. contrôle du stock ;
10. transaction ;
11. API produits ;
12. tests automatisés.

12.3. Architecture attendue

$$\boxed{Request \rightarrow Middleware \rightarrow Controller}$$

$$\boxed{Controller \rightarrow FormRequest}$$

$$\boxed{Controller \rightarrow Service \rightarrow Eloquent}$$

$$\boxed{Controller \rightarrow Blade}$$

ou :

$$\boxed{Controller \rightarrow JSON}$$

13. Proposition de barème pour le mini-projet

Critère	Points
Modélisation et migrations	2
Relations Eloquent	2
CRUD et validation	2
Architecture contrôleurs/services	2
Authentification et autorisation	2
Transactions et gestion du stock	2
API REST	2
Tests automatisés	2
Qualité et lisibilité du code	2
Justification technique / soutenance	2
Total	**20**

14. Grille d'auto-évaluation

Avant de rendre le projet, l'étudiant doit pouvoir répondre **oui** à la majorité des questions suivantes.

1. Mes contrôleurs restent-ils relativement courts ?
2. Les données utilisateur sont-elles validées ?
3. Les opérations sensibles sont-elles autorisées ?
4. Les formulaires utilisent-ils CSRF ?
5. Les relations Eloquent sont-elles définies ?
6. Ai-je vérifié les problèmes N+1 ?
7. Les règles métier complexes sont-elles séparées du contrôleur ?
8. Les opérations multi-tables critiques utilisent-elles une transaction ?
9. Les routes possèdent-elles des noms cohérents ?
10. Les erreurs 404 sont-elles correctement gérées ?
11. L'API utilise-t-elle des codes HTTP adaptés ?
12. Les fonctionnalités importantes possèdent-elles des tests ?
13. Les secrets sont-ils absents du dépôt Git ?
14. Le code est-il compréhensible par un autre développeur ?

15. Synthèse finale

Les premiers TD introduisaient principalement :

$$\boxed{Route \rightarrow Controller \rightarrow View}$$

puis :

$$\boxed{Controller \rightarrow Model \rightarrow Database}$$

La série intermédiaire introduit maintenant :

$$\boxed{Request \rightarrow Middleware \rightarrow Validation}$$

$$\boxed{Authorization \rightarrow Controller \rightarrow Service}$$

$$\boxed{Service \rightarrow Eloquent \rightarrow Transaction}$$

et finalement :

$$\boxed{Application\ Web \quad + \quad API \quad + \quad Tests}$$

Réflexe attendu d'un étudiant de L3

Avant d'ajouter du code, il doit se demander :

1. Quelle est la responsabilité de ce code ?
2. Dans quelle classe doit-il être placé ?
3. Les données ont-elles été validées ?
4. L'utilisateur est-il autorisé ?
5. Une transaction est-elle nécessaire ?
6. Cette requête risque-t-elle de provoquer N+1 ?
7. Comment tester automatiquement ce comportement ?